



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Parallelism III: MIMD / Threads



So far: Matrix Multiplication

[demo]

512 × 512 DGEMM ("double")

- Simple DGEMM

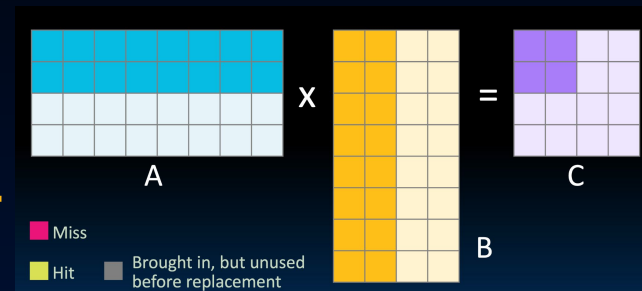
SISD (Single instruction, single data)

- With register keyword
- With cache blocking: transpose
- With cache blocking: tiling

SIMD ("vector instructions")

- SIMD DGEMM
- SIMD with tiling

gcc optimizations: -O0, -O2



lec31_code.zip
A few more edits
pushed this AM,
thanks for your
patience!



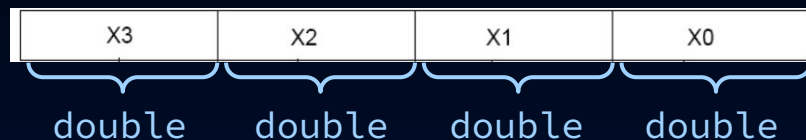
Intel Intrinsics

- SIMD: Intel Intrinsics
-
- MIMD: Multicore
 - Threads
 - Threaded Hello World
 - [Reference] OpenMP

Intel x86 SIMD Extension: Intrinsics

- **SIMD instructions** can often be accessed via **extensions** to a given ISA, e.g., Intel x86.
 - Sometimes known as vector instructions
- **Intrinsics** are C functions and procedures that provide access to assembly language.
 - Import with `<x86intrin.h>`
 - With intrinsics, can program using assembly instructions indirectly.
 - One-to-one correspondence between SIMD extension assembly instruction (e.g., SSE, AVX) and intrinsics.

m256d vector register



[Intel docs](#)

Instead of writing direct assembly code, we will write SIMD code in C using Intel intrinsics.

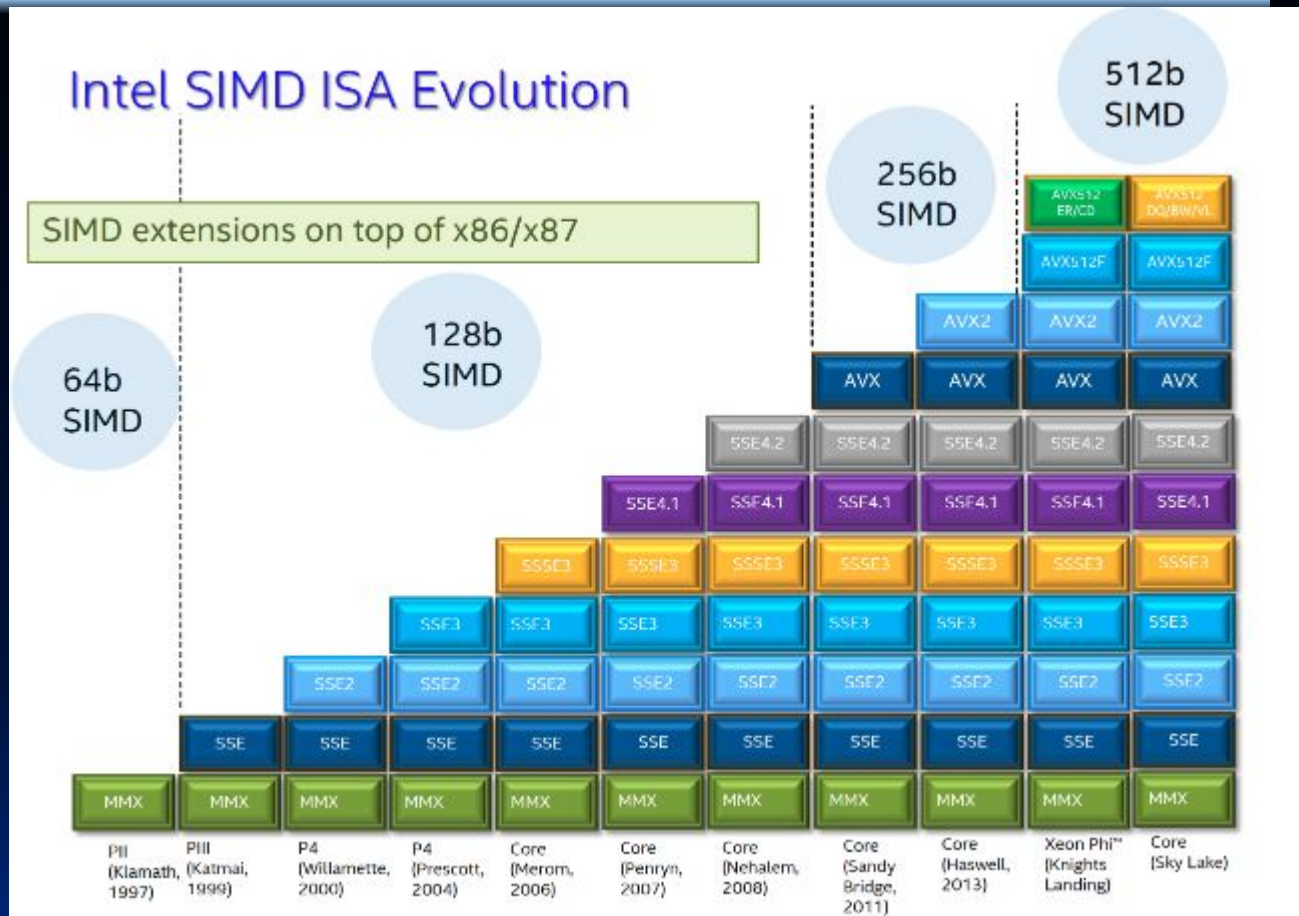
Intel x86 SIMD Evolution (1/2)

Naming of SIMD extensions has changed with functionality:

- New instructions every few years
- New and wider registers

- More parallelism

Intel architectures are generally all backwards compatible! (but docs get very long)





Intel Intrinsics: Instructions and Format

[Intel docs](#)

`_mm<register width>_<instruction>_<data type>`

`__m256i _mm256_add_epi32 (__m256i a, __m256i b);`

256-bit register add 32-bit integer

int	int	int	int	int	int	int	int
-----	-----	-----	-----	-----	-----	-----	-----

`__m128 _mm_load_ps (float const* mem_addr);`

128-bit register load 32-bit float (single-precision)

float	float	float	float
-------	-------	-------	-------

Load 128-bits (of 4 packed single-precision 32-bit floats) from memory into dst. mem_addr must be aligned on a 16-byte boundary.



Demo

`sse_example()`

In plain english, what is each intrinsic call doing?

Intel docs



Yan, SP26

Back to matmul: Two Observations, Made in Parallel

- Element-wise multiplication (of row of A, column of B) can be computed independently
 - **One FP instruction:** element-wise multiplication
- Different elements of matrix C can be computed independently
 - **Separate sequences** of `fmul` + `fadd`

Can we
parallelize even
more?

Perhaps...parallelism?

Back to matmul: Can we go even faster?

```
/* openmp, double */
matrix_d_t *dgemm_openmp(matrix_d_t *A, matrix_d_t *B) {
    if (A->ncols != B->nrows) return NULL;
    matrix_d_t *C = init_mat_d(A->nrows, B->ncols);

    #pragma omp parallel for
    for (int i = 0; i < A->nrows; i++) {
        for (int j = 0; j < B->ncols; j++) {
            double sum = 0;
            for (int k = 0; k < A->ncols; k++) {
                sum += A->data[i*A->ncols+k]*B->data[k*B->ncols+j];
            }
            C->data[i*C->ncols+j] = sum;
        }
    }
    return C;
}
```

Introducing: **Thread-Level Parallelism**. Use OpenMP to leverage MIMD hardware!

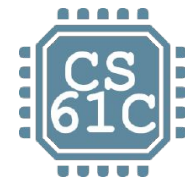




CS 61C Industry & Research Panel

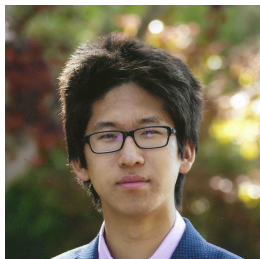
RRR week May 5 (Tue), 2-3pm, Wozniak Lounge

bit.ly/61c-panel



Nicolas Rakela

- BS EECS'26 - UCB
- MS EECS'27 - UCB
- SLICE/BWRC Lab
- EECS 16A, EECS 16B, EECS 151, Tapeout TA
- Apple Intern ('25, '26)
- Apple MS Scholar



Jim Fang

- BS EECS'27 - UCB
- SLICE/BWRC Lab
- Former CS 61C TA
- Tapeout Head TA
- IEEE Chair
- Google Student Researcher ('25-'26)



Professor Sagar Karandikar

- BS EECS'15 - UCB
- PhD EECS'24 - UCB
- SLICE/BWRC Lab
- Former 6x CS 61C TA (4x as Head TA) and Lecturer
- Google Student/Faculty Researcher ('20-'25)



Professor Chris Fletcher

- BS EECS'10 - UCB
- PhD EECS'16 - MIT
- SLICE/BWRC Lab
- Former UIUC Professor
- NVIDIA Architecture Research Group ('16-'17)



Andre Green

- BS EE'23 - Penn
- MS EECS'25 - UCB
- PhD EECS'28 - UCB
- SLICE/BWRC Lab
- Former CS61C TA
- IBM Intern ('22-'23)
- NVIDIA Architecture Research Group ('25-)



Kelly Tou

- BS EECS'27 - UCB
- Former SWARM Lab
- SLICE/BWRC Lab
- EECS 151 TA
- 2x Apple NSI Design Winner
- Apple Intern ('25, '26)



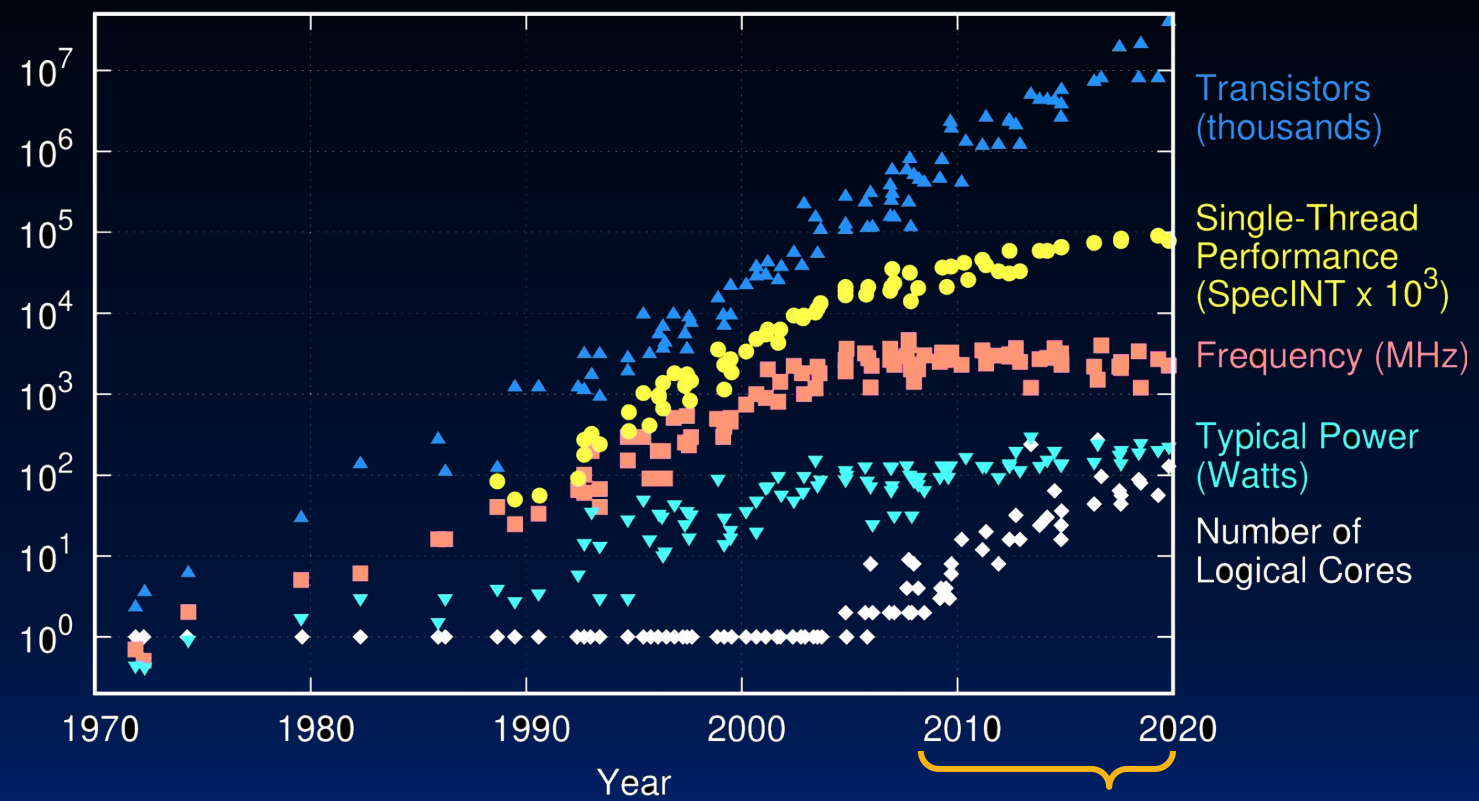
MIMD: Multicore

- SIMD: Intel Intrinsics
-
- MIMD: Multicore
- Threads
- Threaded Hello World
- [Reference] OpenMP



Processor Performance: Trends

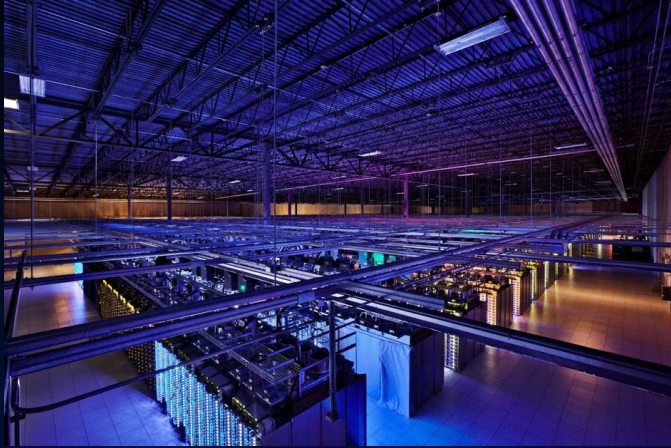
48 Years of Microprocessor Trend Data



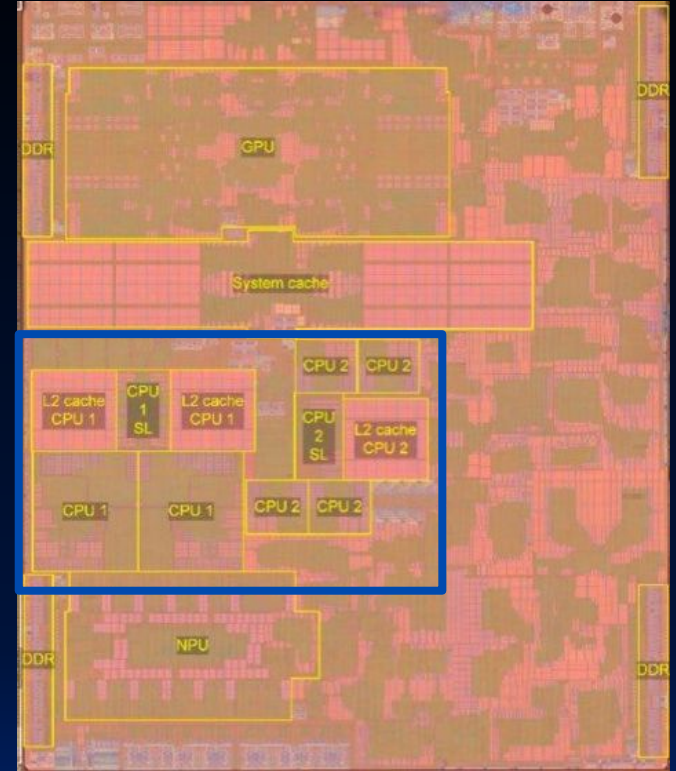
Karl Rupp

Yan, SP26

Parallel Computer Architectures



warehouse-scale
computers

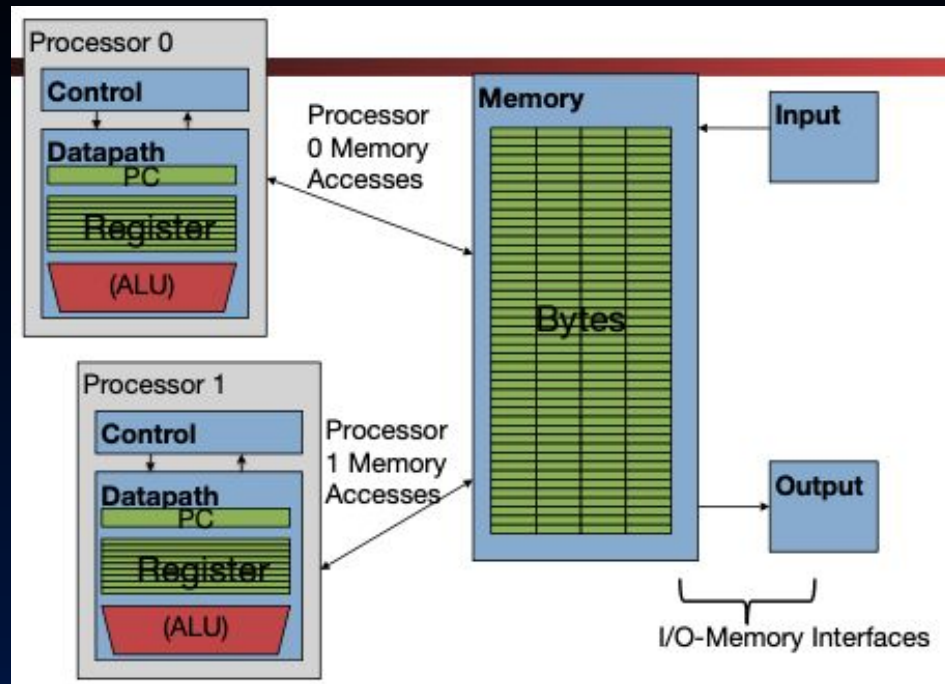


multicore

Apple M14 ([link](#))

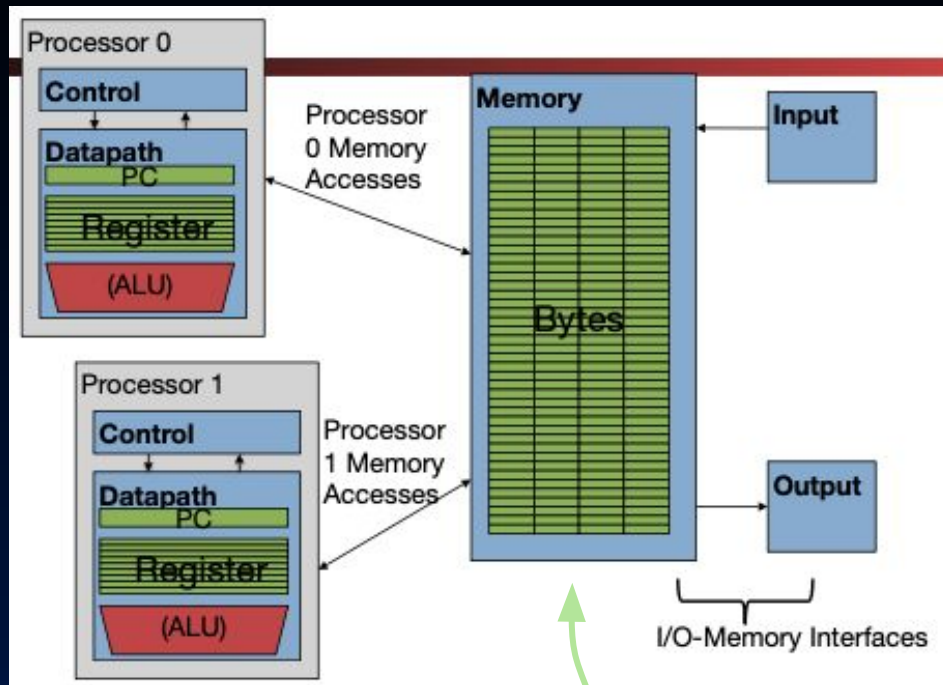
Yan, SP26

- A **multicore processor** contains multiple processors ("cores") in a single integrated circuit.
- Execution Model:
 - (1) Each processor has executes an **independent stream of instructions**.
 - Also separate: high-level caches (L1, L2)



FA21 L22

- A **multicore processor** contains multiple processors ("cores") in a single integrated circuit.
- Execution Model:
 - (1) Each processor has executes an **independent stream of instructions**.
 - (2) All processors access the same **shared memory**.
 - Shared: DRAM, perhaps L3 cache
 - Communicate via shared memory by storing to/loading from common locations.



FA21 L22

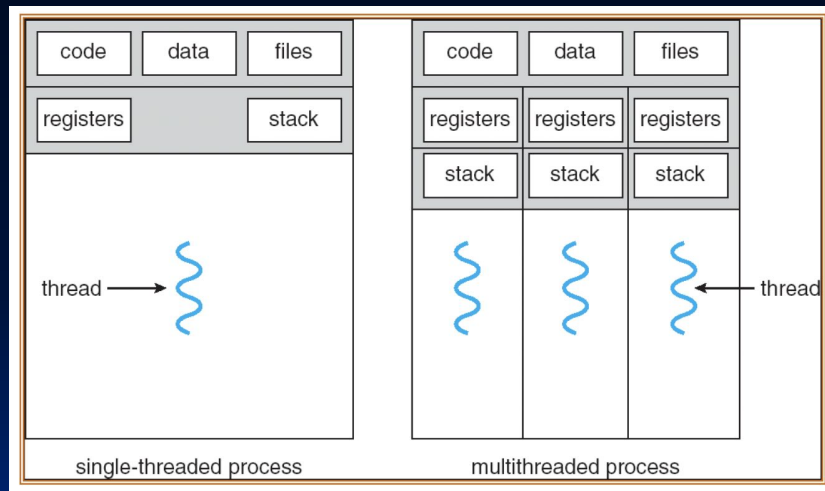


Threads

- SIMD: Intel Intrinsics
-
- MIMD: Multicore
- Threads
- Threaded Hello World
- [Reference] OpenMP

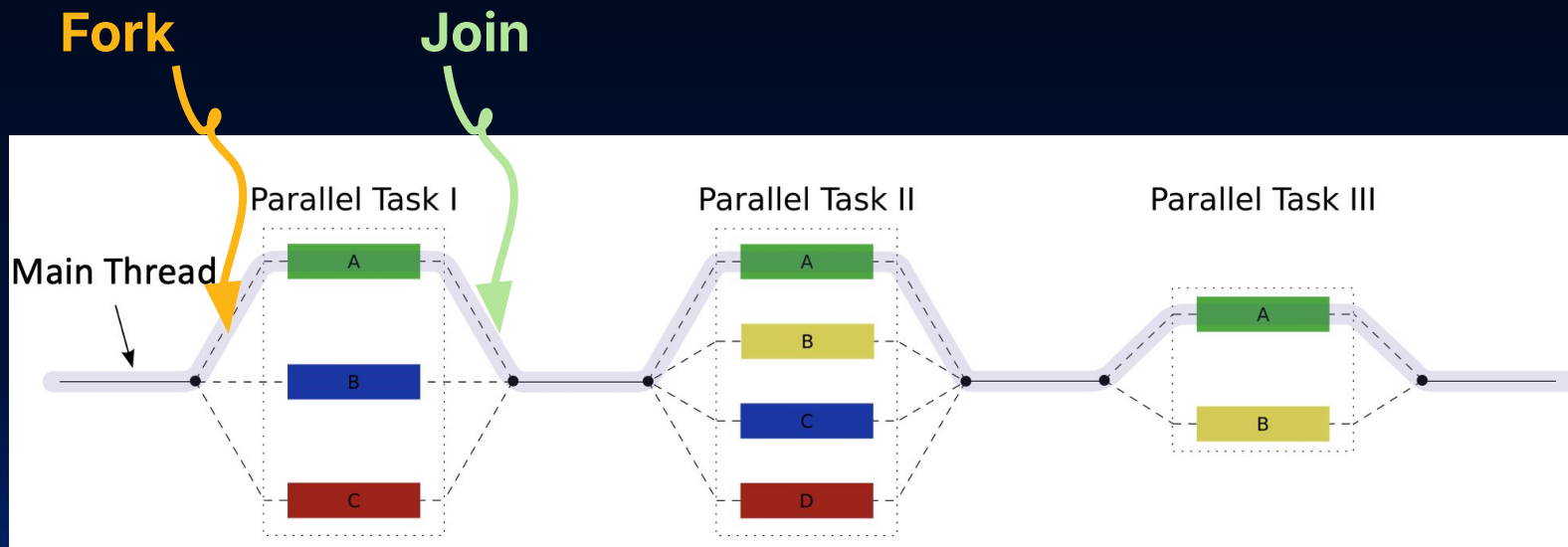
Threads

- **Threads** (short for **threads of execution**) is a single stream of instructions.
- Each thread has:
 - Values of its own registers (including stack pointer)
 - Value of its own program counter (PC)
 - Shared memory (heap, global variables) with other threads
- Within a given program's process, threads can run **concurrently**.



The Fork-Join Model

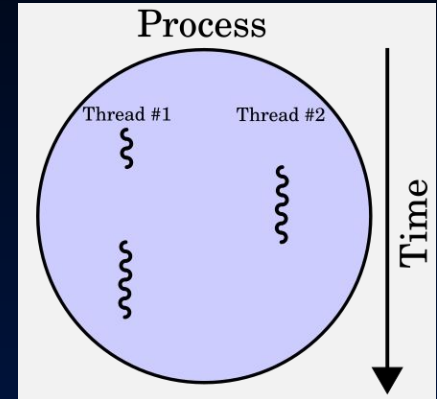
- **Fork-Join model:** A program / process can split, or fork itself into separate threads, which can (in theory) **execute concurrently**.
 - Main thread executes sequentially until first parallel task region.





Executing threads on hardware

- Each core provides one (or more) **hardware threads** that actively execute instructions.
- The Operating System multiplexes multiple **software threads** onto the available hardware threads.
 - def "Multiplex" combine multiple channels into one shared channel (e.g., MUX).
 - Only those mapped onto hardware threads are executing; **all others are waiting**.



A process with two (software) threads of execution, perhaps mapped to a single hardware thread [\[wiki\]](#)

Yan, SP26

Modern Machines: Multicore

```
$ sysctl -a | grep "logicalcpu\|physicalcpu" # on laptop (Apple M4)
```

```
hw.perflevel0.physicalcpu: 4  
hw.perflevel0.logicalcpu: 4  
hw.perflevel1.physicalcpu: 6  
hw.perflevel1.logicalcpu: 6  
hw.physicalcpu: 10  
hw.logicalcpu: 10
```

For each system,
how many
hardware threads?
How many cores?

```
$ lscpu # on hive (Intel machine)
```

```
CPU(s): 12  
On-line CPU(s) list: 0-11  
Vendor ID: GenuineIntel  
Model name: Intel(R) Core(TM) i7-8700T CPU @ 2.40GHz  
CPU family: 6  
Model: 158  
Thread(s) per core: 2  
Core(s) per socket: 6  
Socket(s): 1
```



Yan, SP26



Modern Machines: Multicore

\$ sysctl -a | grep "logicalcpu\|physicalcpu" # on laptop (Apple M4)

```
hw.perflevel0.physicalcpu: 4
hw.perflevel0.logicalcpu: 4
hw.perflevel1.physicalcpu: 6
hw.perflevel1.logicalcpu: 6
hw.physicalcpu: 10
hw.logicalcpu: 10
```

"10 CPUs"

\$ lscpu # on hive (Intel machine)

```
CPU(s): 12
On-line CPU(s) list: 0-11
Vendor ID: GenuineIntel
Model name: Intel(R) Core(TM) i7-8700T CPU @ 2.40GHz
CPU family: 6
Model: 158
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 1
```

"12 logical CPUs"
(Intel SMT: hardware
multithreading)

"6 physical CPUs"

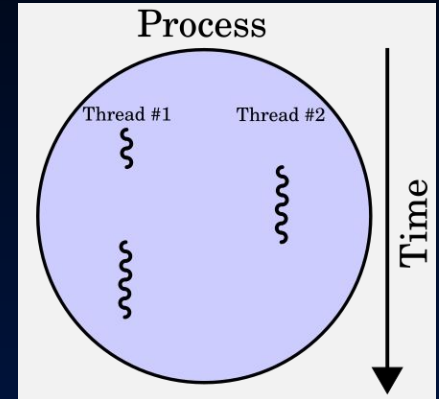


OS Context Switches

- The OS effectively gives the illusion of many active threads by time-multiplexing software threads onto hardware threads via a process called **context switches**.
 - If one thread is blocked (e.g., waiting for network access/user input), switch to another thread
 - A thread should not “notice” it was paused

Context switching is an OS process.

Because of OS context switching, a single CPU can still execute multiple threads.

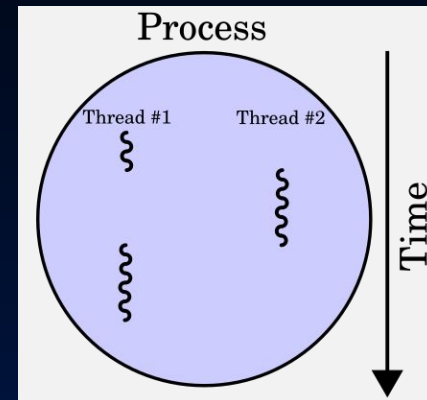


A process with two (software) threads of execution, perhaps mapped to a single hardware thread [\[wiki\]](#)

Yan, SP26

OS Context Switches

- The OS effectively gives the illusion of many active threads by time-multiplexing software threads onto hardware threads via a process called **context switches**.
 - If one thread is blocked (e.g., waiting for network access/user input), switch to another thread
 - A thread should not “notice” it was paused
- To remove old SW thread from HW thread:
 - Interrupt execution
 - Save registers to memory, including PC
- To activate new SW thread onto HW thread:
 - Load register values for new thread, including PC
 - Transfer control to new thread



A process with two (software) threads of execution, perhaps mapped to a single hardware thread [\[wiki\]](#)

Yan, SP26



Threaded Hello World

- SIMD: Intel Intrinsics
-
- MIMD: Multicore
- Threads
- Threaded Hello World
- [Reference] OpenMP

Thoughts about Threads

"Although threads seem to be a small step from sequential computation, in fact, they represent a huge step. They discard the most essential and appealing properties of sequential computation: understandability, predictability, and determinism. **Threads, as a model of computation, are wildly nondeterministic**, and the job of the programmer becomes one of pruning that nondeterminism."



Professor Emeritus
Edward Lee
UC Berkeley

Edward A. Lee. "The Problem with Threads."

Technical Report No. UCB/EECS-2006-1. January 2006.

<http://www.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-1.html>

See also: Computer 39, 5 (May 2006), 33–42. <https://doi.org/10.1109/MC.2006.180>



OpenMP API

- **OpenMP** is an API (Application Programming Interface) with language **extensions for C, C++, and Fortran**.
 - Enables **multi-threaded, shared memory parallelism**.
 - Generally follows the fork-join model
 - "Open Multi-Processing"
- Portable, standardized, and easy to compile:
 - `#include <omp.h>`
 - `gcc -fopenmp foo.c`
- Key ideas: (more next time/in lab)
 - Shared vs. private variables
 - OpenMP directives for:
 - Parallelization and work sharing
 - Synchronization

```
1  #include <stdio.h>
2  #include <omp.h>
3  int main() {
4      /* Fork team of threads with private variable tid */
5      #pragma omp parallel
6      {
7          int tid = omp_get_thread_num(); /* get thread id */
8          printf("Hello World from thread = %d\n", tid);
9          /* Only main thread does this */
10         if (tid == 0) {
11             printf("Number of threads = %d\n",
12                   omp_get_num_threads());
13         }
14     } /* All threads join main and terminate */
15     return 0;
16 }
17 }
```

```
1  #include <stdio.h>
2  #include <omp.h>
3  int main() {
4      /* Fork team of threads with private variable tid */
5      #pragma omp parallel
6      {
7          int tid = omp_get_thread_num(); /* get thread id */
8          printf("Hello World from thread = %d\n", tid);
9          /* Only main thread does this */
10         if (tid == 0) {
11             printf("Number of threads = %d\n",
12                    omp_get_num_threads());
13         }
14     } /* All threads join main and terminate */
15     return 0;
16 }
17 }
```

} parallel region
executed by
each subthread
(with OpenMP
API)



Thread-Based Parallelism: Assumptions (1/2)

1. Explicit programming model with full programmer control over parallelization

- Pros:

- Compiler directives are simple and easy to use
- Legacy serial code does not need to be rewritten

- Cons:

- Compiler must support OpenMP (e.g. gcc 4.2)
- Amdahl's law is gonna get you after not too many cores...

2. Multiple threads in a **shared memory environment**

What does this mean?

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  /* Fork team of threads with private variable tid, shared variable x */
5  int main() {
6      int x = 0;                                /* shared variable */
7      #pragma omp parallel
8      {
9          int tid = omp_get_thread_num(); /* get thread id */
10         x++;
11         printf("Hello World from thread = %d, x = %d\n", tid, x);
12         /* Only main thread does this */
13         if (tid == 0) {
14             printf("Number of threads = %d\n",
15                 /* ... */
16             }
17     } /* All threads join main and terminate */
18     return 0;
19 }
```

⚠️ race condition! ⚠️

What do you think the last
printed value of x is?





Thread-Based Parallelism: Assumptions (2/2)

1. Explicit programming model with full programmer control over parallelization
2. Multiple threads in a **shared memory environment**
 - Pros:
 - Takes advantage of shared memory, programmer need not worry (that much) about data placement
 - Cons:
 - Code can only be run in shared memory environments
 - **Synchronizing use of shared resources is hard** (more next time)



Timing Program Execution

[reference]

- `clock()` from `<time.h>` measures active CPU time.
 - In other words, **combined CPU time** (in cycles) across all threads!
- `omp_get_wtime()` measures **wall time** (in seconds) with respect to one thread
 - Per-thread—i.e., not consistent across threads!

```
#include <omp.h>
int main(...) {
    double start, end;
    start = omp_get_wtime();
    /* ... work to be timed ... */
    end = omp_get_wtime();
    printf("Work took %f seconds\n", end - start);
}
```




Data Races

- SIMD: Intel Intrinsics
-
- MIMD: Multicore
- Threads
- Threaded Hello World
- [Reference] OpenMP

Suppose we run the below code on 4 threads.

```
int x = 0;
#pragma omp parallel
{
    x = x + 1;
}
```

What are possible values of x after running this code?
Select all that apply.

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4
- F. 5 or more



What are possible values of x after running this code? Select all that apply.

Suppose we run the below code on 4 threads.

```
int x = 0;
#pragma omp parallel
{
    x = x + 1;
}
```

A. 0

☐

0

B. 1

☐

0

C. 2

☐

0

D. 3

☐

0

E. 4

☐

0

F. 5 or more

☐

0



SP21

Data Race

- Two memory accesses form a **data race** if:
 - They are from different threads to the same location
 - At least one is a write, and
 - They occur one after another.
- Remember thread model: **shared memory**.
 - For a given thread, these two operations don't necessarily happen together...!
 - Read current value of x
 - Write new value of x
- Not a data hazard...!
 - Data hazard: Sequential instructions have data dependencies during concurrent execution (instruction-level parallelism via pipelining).
 - Here, even with no ILP, can have **nondeterministic results**. This results from lack of synchronization on which thread accesses memory first.

Data Race: RISC-V Instructions

- Instructions from different threads have their execution on the CPU **interleaved**.
- Assume (for ease of analysis):
 - Non-pipelined, single-cycle datapath → all **atomic instructions**, meaning that no nothing else interposes itself while the instruction is executing.

```
// four threads  
int x = 0;  
#pragma omp parallel  
{  
    x = x + 1;  
}
```

sw x0 0(sp)

lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)

load
addi
store

load
addi
store

load
addi
store

load
addi
store



Data Race Case 1: All threads run sequentially

- **Orange** thread load read x: 0 sw x0 0(sp)
 - **Orange** thread store write x: 1 load
 - **Green** thread load read x: 1 addi
 - **Green** thread store write x: 2 store
 - **Blue** thread load read x: 2 load
 - **Blue** thread store write x: 3 addi
 - **Purple** thread load read x: 3 store
 - **Purple** thread store write x: 4 load
- addi
store
load
addi
store
- Final value of x: 4

Data Race Case 2: Threads perfectly interleaved

• Orange thread load	read	x: 0	sw x0 0(sp)
• Green thread load	read	x: 0	load
• Blue thread load	read	x: 0	load
• Purple thread load	read	x: 0	load
• Orange thread store	write	x: 1	load
• Green thread store	write	x: 1	addi
• Blue thread store	write	x: 1	addi
• Purple thread store	write	x: 1	addi
			store
			store
			store
			store

Final value of x: 1

Data Race Case 3: Orange Store Happens Last

• Orange thread load	read	x: 0	sw x0 0(sp)
• Green thread load	read	x: 0	load
• Green thread store	write	x: 1	addi
• Blue thread load	read	x: 1	load
• Blue thread store	write	x: 2	addi
• Purple thread load	read	x: 2	store
• Purple thread store	write	x: 3	load
• Orange thread store	write	x: 1	addi
			store
			load
			addi
			store
			store

Final value of x: 1





Data Race: Correctness

- 105* possible instruction orderings to generate final value of x!
 - Can't go above 4 (only four "+1" instructions overall)
 - Can't go below 1 (smallest load value is 0, therefore smallest last store value is 1)
 - Can get **any value between 1 and 4**
- Formally, a multithreaded program is only considered correct if ANY interlacing of threads yield the same result.
 - Here, we have an incorrect program!
 - But if each thread works on independent data (no thread accesses same data location another thread wrote to), you can guarantee correctness.

*speaker notes

The hardest part of multithreading is understanding and maintaining correctness, possibly through thread coordination.
More next time!



[Reference] OpenMP

- SIMD: Intel Intrinsics
-
- MIMD: Multicore
- Threads
- Threaded Hello World
- [Reference] OpenMP



OpenMP API

- **OpenMP** is an API (Application Programming Interface) with language **extensions for C**, C++, and Fortran.
 - Enables **multi-threaded, shared memory parallelism**.
 - Generally follows the fork-join model
 - "Open Multi-Processing"
- Portable, standardized, and easy to compile:
 - `#include <omp.h>`
 - `gcc -fopenmp foo.c`
- Key ideas:
 - Shared vs. private variables (today)
 - OpenMP directives for:
 - Parallelization and work sharing (today)
 - Synchronization (next time)



OpenMP Extends C with Pragmas

- **Pragmas** are a preprocessor mechanism C provides for language extensions.
 - Commonly implemented pragmas not covered in 61C: structure packing, symbol aliasing, floating point exception modes.
- Good mechanism for OpenMP because compilers that don't recognize a pragma are supposed to ignore them.
 - Even with embedded pragmas, can run on sequential computer!

- Basic OpenMP construct for parallel regions (right):
 - Each thread runs the instructions within the block
 - Thread scheduling is non-deterministic

```
#pragma omp parallel
{
    /* code goes here */
}
```



OpenMP: Threads

- OpenMP creates as many threads as specified in the environment variable **OMP_NUM_THREADS**.
 - Set this variable to max number of threads you want to use
 - Generally, default: (# physical cores) * (# threads/core)...(more later)
 - Hive machines (use `lscpu`): $6 * 2 =$ **12 threads**
- OpenMP threads are OS (software) threads, which are then multiplexed onto available hardware threads. (more later)

OpenMP Intrinsic	Description
<code>omp_set_num_threads(x);</code>	Set number of threads to x.
<code>num_th = omp_get_num_threads();</code>	Get number of threads.
<code>th_ID = omp_get_thread_num();</code>	Get Thread ID number.



OpenMP: Shared vs. Private Variables

[OpenMP docs](#)

- **Shared variables:** all threads read/write the same variable.
 - Variable declared outside of parallel region
 - Heap-allocated variables
 - Static variables
- **Private variables:** Each thread has its own copy of the variable.
 - Variables declared inside parallel region (recall separate stack frames)

```
int var1, var2;
char *var3 = malloc(...);
#pragma omp parallel private(var2)
{
    int var4;
    // var1 shared (default)
    // var2 private
    // var3 shared (heap)
    // var4 private (thread's stack)
    ...
}
```

Heap memory is shared across threads.



OpenMP: What Kind of Threads?

[more later]

- OpenMP threads are **operating system threads**, i.e., **software threads**.
 - OS will multiplex requested OpenMP threads onto available hardware threads.
 - Hopefully each gets a real hardware thread to run on, so no OS-level time-multiplexing.
- But other tasks on machine can also use hardware threads!
 - And you may want more threads than hardware if you have a lot of I/O so that while waiting for I/O other threads can run.
 - Be “careful” when timing results! You may inadvertently measure shared machine workload, particularly during peak times.

(more later)

Many Parallel Programming Languages Exist...

[out of scope]

- There is no universal solution to parallel programming.
 - Needs are very problem-specific: scientific computing/ML, web servers, I/O, ...

- OpenMP: threads, fork-join model
 - Different models for different applications (e.g., message passing for process-level parallelism, with concurrent independent tasks)

ActorScript	Concurrent Pascal	JoCaml	Orc
Ada	Concurrent ML	Join	Oz
Afnix	Concurrent Haskell	Java	Pict
Alef	Curry	Joule	Reia
Alice	CUDA	Joyce	SALSA
APL	E	LabVIEW	Scala
Axum	Eiffel	Limbo	SISAL
Chapel	Erlang	Linda	SR
Cilk	Fortan 90	MultiLisp	Stackless Python
Clean	Go	Modula-3	SuperPascal
Clojure	Io	Occam	VHDL
Concurrent C	Janus	occam- π	XC



For Loop Work-Sharing Example

- SIMD: Intel Intrinsics
-
- MIMD: Multicore
- Threads
- Threaded Hello World
- [Reference] OpenMP

Parallelizing Loop Work

- Problem: You have to do some work over an array of 2^{27} numbers, with 12 people. How do you split the work?
- Assumptions:
 - We need to decide this before we run the code!
 - Each element of the array is independent, so we can do this in any order
 - The threads are about equally fast at work, so we want to assign each of them ~11 million numbers

```
#define LENGTH (1 << 27)
int main(...) {
    char *arr = malloc(sizeof(char) * LENGTH);
    /* how to parallelize this loop? */
    for (int i = 0; i < LENGTH; i++) {
        arr[i] = ...;
    }
}
```

```
/* A. */
#pragma omp parallel
{
    for (int i = 0;
        i < LENGTH;
        i++) {
        arr[i] = ...;
    }
}
```

```
/* B. */
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int num_threads = omp_get_num_threads();
    for (int i = tid;
        i < LENGTH;
        i += num_threads) {
        arr[i] = ...;
    }
}
```

```
/* C. */
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int num_threads = omp_get_num_threads();
    int thread_start = tid*LENGTH/num_threads;
    int thread_end = (tid+1)*LENGTH/num_threads;
    for (int i = thread_start;
        i < thread_end; i++) {
        arr[i] = ...;
    }
}
```

```
/* D. */
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0;
        i < LENGTH;
        i++) {
        arr[i] = ...;
    }
}
```

```
/* A. */
#pragma omp parallel
{
    for (int i = 0;
        i < LENGTH;
        i++) {
        arr[i] = ...;
    }
}
```

```
/* B. */
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int num_threads = omp_get_num_threads();
    for (int i = tid;
        i < LENGTH;
        i += num_threads) {
        arr[i] = ...;
    }
}
```

```
/* C. */
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int num_threads = omp_get_num_threads();
    int thread_start = tid*LENGTH/num_threads;
    int thread_end = (tid+1)*LENGTH/num_threads;
    for (int i = thread_start;
        i < thread_end; i++) {
        arr[i] = ...;
    }
}
```

```
/* D. */
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0;
        i < LENGTH;
        i++) {
        arr[i] = ...;
    }
}
```





Which Runs Fastest?

[demo]

```
/* A. */
#pragma omp parallel
{
    for (int i = 0;
        i < LENGTH;
        i++) {
        arr[i] = ...;
    }
}
```

duplicates work

```
/* B. */
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int num_threads = omp_get_num_threads();
    for (int i = tid;
        i < LENGTH;
        i += num_threads) {
        arr[i] = ...;
    }
}
```

Code Drive

"interweaves" array access between threads

```
/* C. */
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int num_threads = omp_get_num_threads();
    int thread_start = tid*LENGTH/num_threads;
    int thread_end = (tid+1)*LENGTH/num_threads;
    for (int i = thread_start;
        i < thread_end; i++) {
        arr[i] = ...;
    }
}
```

"chunks" array sections

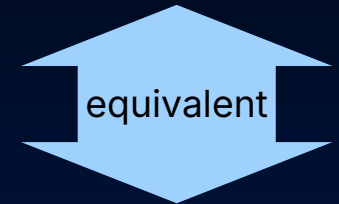
```
/* D. */
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0;
        i < LENGTH;
        i++) {
        arr[i] = ...;
    }
}
```

Like C, but planned via OpenMP



- `#pragma omp for`
 - must be written inside an already existing parallel segment.
 - If a parallel segment consists only of one for loop, we can combine the two declarations with `#pragma omp parallel for`.
- Must have relatively simple “shape” for an OpenMP-aware compiler to be able to parallelize it
 - Necessary for the run-time system to be able to determine how many of the loop iterations to assign to each thread
- No premature exits from the loop allowed
 - i.e. No `break`, `return`, `exit`, `goto`

```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0;
        i < LENGTH;
        i++) {
        arr[i] = ...;
    }
}
```



```
#pragma omp parallel for
for (int i = 0;
    i < LENGTH;
    i++) {
    arr[i] = ...;
}
```